

Desarrollo de Aplicaciones Web con Java

De los servlets a los frameworks — guía orientada a un proyecto real

Servlets · JSP · JSTL · EL | Spring Boot · Jakarta Server Faces (JPA, JTA)

UTEQ · Ingeniería de Software · Dr. Gleiston Guerrero Ulloa, Ph.D.

CÓMO ESTÁ ORGANIZADA

Dos partes, un mismo proyecto

Parte I — Tecnología base

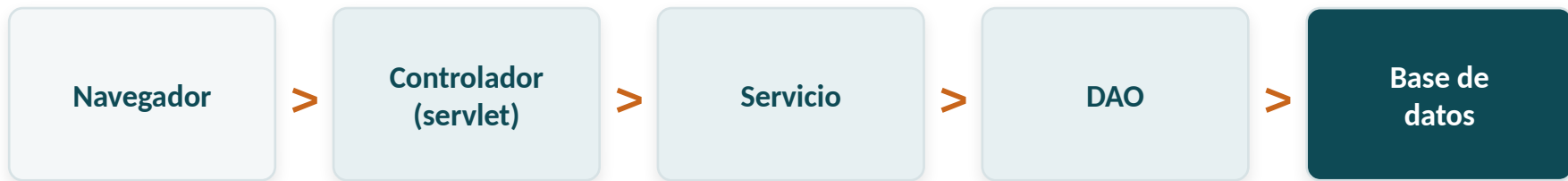
Servlets (controlador), JSP (vista), EL y JSTL. Arquitectura por capas y MVC. Sesiones, login y control de acceso. CRUD, reportes y reportes impresos. Construimos la app a mano para entender cada pieza.

Parte II — Frameworks

Spring Boot (JPA, Spring Data, Spring Security) y Jakarta Server Faces (JPA, JTA, CDI). El mismo problema, resuelto con frameworks que reducen el código repetitivo.

Método: cada tema plantea su problema → lo explica → lo resuelve con un ejemplo pequeño y completo.

Petición-respuesta y arquitectura por capas



La vista (JSP con JSTL y EL) presenta el resultado al navegador

- HTTP no tiene estado: el servidor no recuerda al usuario entre peticiones (lo resuelven las sesiones).
- El contenedor de servlets (Tomcat) recibe la petición, ejecuta tu código y devuelve la respuesta.
- Cada capa tiene una responsabilidad y solo conoce a la de abajo: así el proyecto crece sano.

Servidor, empaquetado y versiones

PROBLEMA A RESOLVER

Para ejecutar código Java que atienda al navegador necesitamos un servidor que lo aloje y un proyecto bien empaquetado. ¿Cómo lo preparamos?

Lo esencial

JDK LTS (21/25) · IntelliJ IDEA · Apache Tomcat 11.
Empaquetado WAR. Las JSP van en WEB-INF para no exponerlas. Paquetes jakarta.* (no javax.*).

Versiones de referencia (jun. 2026)

Java 25 LTS (y 21)
Tomcat 11 · Jakarta EE 11
Jakarta Servlet 6.1
Spring Boot 4 · Faces 4.1
PostgreSQL 18

PARTE I

Tecnología base

Servlets, JSP, JSTL y el lenguaje de expresiones

El controlador del lado del servidor

PROBLEMA A RESOLVER

Cuando el navegador pide una dirección, ¿cómo hacemos que un trozo de código Java se ejecute en el servidor y devuelva una respuesta a medida?

- Hereda de `HttpServlet`; `doGet` atiende GET y `doPost` atiende POST.
- `@WebServlet` asocia el servlet a una URL.
- En MVC, el servlet es el controlador.

HolaServlet.java

```
@WebServlet("/hola")
public class HolaServlet
    extends HttpServlet {
    protected void doGet(
        HttpServletRequest req,
        HttpServletResponse resp) {
        resp.setContentType(
            "text/html; charset=UTF-8");
        resp.getWriter()
            .println("<h1>Hola</h1>");
    }
}
```

SECCIÓN 3 — SERVLETS

Leer datos y delegar en la vista

PROBLEMA A RESOLVER

El usuario envía un formulario. ¿Cómo lee el servlet esos datos y deja que la vista los muestre?

SaludarServlet.java (fragmento)

```
String nombre = req.getParameter("nombre");    // dato del formulario
if (nombre == null || nombre.isBlank()) nombre = "invitado"; // validar
req.setAttribute("nombre", nombre);           // pasar a la vista
req.getRequestDispatcher("/WEB-INF/vistas/saludo.jsp")
    .forward(req, resp);                       // MVC: servlet -> JSP
```

Regla: validar SIEMPRE la entrada en el servidor; la validación del navegador es solo comodidad.

La vista

PROBLEMA A RESOLVER

Generar HTML con `out.println` dentro del servlet es tedioso y mezcla lógica y presentación. ¿Cómo los separamos?

saludo.jsp

```
<%@ page contentType=
    "text/html; charset=UTF-8" %>
<h1>Hola, ${nombre}</h1>
```

Sin código Java en la vista

JSP es HTML con datos dinámicos: el contenedor la convierte en un servlet, pero tú escribes algo parecido a HTML. Regla moderna: nada de scriptlets `<% %>`; solo EL (`${...}`) y etiquetas JSTL. La lógica vive en el servlet y el servicio.

El lenguaje de expresiones

PROBLEMA A RESOLVER

Dentro de la JSP necesitamos leer datos y propiedades de objetos sin escribir código Java. ¿Cómo?

Requiere JavaBeans

Para que `${objeto.propiedad}` funcione, el objeto debe tener getters/setters. Por eso el modelo se programa con clases JavaBean, no con record.

EL en la JSP

```
${estudiante.nombre}  
    // llama a getNombre()  
${sessionScope.usuario}  
    // dato de la sesion  
${2 + 3}    // evalua
```

EL distingue ámbitos: `requestScope`, `sessionScope`, `applicationScope`.

Lógica de presentación sin scriptlets

PROBLEMA A RESOLVER

En la vista necesitamos repetir filas (una por estudiante) y condicionar contenido, sin volver a escribir código Java. ¿Cómo?

c:forEach repite · c:if / c:choose condicionan · c:url para enlaces.

c:out escapa el texto: defensa anti-XSS, igual que el escapado manual.

lista.jsp (fragmento)

```
<%@ taglib prefix="c"
    uri="jakarta.tags.core" %>
<c:forEach var="e"
    items="${estudiantes}">
    <td><c:out value="${e.nombre}"/>
</c:forEach>
```

Capas y patrón MVC

PROBLEMA A RESOLVER

Si mezclamos acceso a datos, lógica y presentación en cada servlet, el proyecto se vuelve inmanejable. ¿Cómo lo organizamos?

Modelo

JavaBean: representa los datos

DAO

Acceso a datos (JDBC)

Servicio

Lógica y validaciones

Controlador

Servlets: coordinan

Vista

JSP + JSTL + EL

Programar con clases bien definidas en cada capa es lo que mantiene el orden al crecer.

Inicio de sesión y control de acceso

PROBLEMA A RESOLVER

Necesitamos un inicio de sesión que verifique credenciales y, además, impedir el acceso a las páginas privadas a quien no ha iniciado sesión.

Idea del login

```
// tras validar credenciales (BCrypt):  
HttpSession s = req.getSession();  
s.setAttribute("usuario", usuario);  
resp.sendRedirect("../estudiantes");
```

AutenticacionFilter.java

```
@WebFilter("/estudiantes")  
// si hay usuario en sesion:  
    chain.doFilter(req, resp);  
// si no:  
    resp.sendRedirect("../login");
```

Crear, listar, editar y eliminar

PROBLEMA A RESOLVER

El corazón de la aplicación: gestionar registros con un DAO, un servlet controlador y vistas JSP.

- Un servlet enruta las acciones con un parámetro accion.
- Marcadores ? en todo SQL: anti-inyección.
- Tras guardar/eliminar → redirige (patrón PRG).

DAO con sentencias preparadas

```
String sql = "INSERT INTO estudiantes"
            + " (nombre,correo,carrera) VALUES (?, ?, ?)";
PreparedStatement ps =
    con.prepareStatement(sql); // anti-
    inyeccion
ps.setString(1, e.getNombre());
ps.executeUpdate();
```

Información resumida

PROBLEMA A RESOLVER

La dirección necesita datos agregados: por ejemplo, cuántos estudiantes hay por carrera.

Consulta de agregación (SQL)

```
SELECT carrera, COUNT(*) AS total  
FROM estudiantes  
GROUP BY carrera  
ORDER BY carrera;
```

Mismo patrón que el listado

El DAO ejecuta la consulta GROUP BY y devuelve los totales; el servlet los pasa a una JSP que los recorre con `c:forEach` (para un mapa, la variable expone key y value). La diferencia con el listado está solo en la consulta.

Reportes impresos

PROBLEMA A RESOLVER

La secretaría necesita un documento para imprimir o archivar, idealmente en PDF. ¿Cómo lo generamos?

Camino sencillo: imprimir HTML

```
@media print {  
  nav, .botones { display:none; }  
  body { font-size: 12pt; }  
}  
// el navegador: Guardar como PDF
```

PDF en el servidor (OpenPDF)

```
resp.setContentType("application/pdf");  
PdfWriter.getInstance(doc,  
    resp.getOutputStream());  
doc.add(new Paragraph("..."));  
doc.close();
```

El problema completo

Sistema de Gestión de Estudiantes

Reunimos todos los subproblemas en una sola aplicación.

- 1 **Inicio de sesión** — valida usuario y contraseña
- 2 **Control de acceso** — solo autenticados entran a la gestión
- 3 **CRUD** — crear, listar, editar y eliminar
- 4 **Reportes** — total de estudiantes por carrera
- 5 **Reportes impresos** — descargar el reporte en PDF

Cada requisito y sus piezas

Requisito	Piezas que lo resuelven
Inicio de sesión	LoginServlet + login.jsp + UsuarioServicio (hash BCrypt)
Control de acceso	AutenticacionFilter sobre rutas privadas + la sesión
CRUD	EstudianteServlet + EstudianteDAO + lista.jsp, form.jsp
Reportes	contarPorCarrera() + reporte.jsp con c:forEach
Reportes impresos	@media print y ReportePdfServlet (OpenPDF)

Todo el código ya está escrito en las secciones previas: aquí solo se ve el conjunto.

PARTE II

Frameworks del backend

Spring Boot y Jakarta Server Faces (JPA, JTA)

SECCIÓN 13

¿Por qué usar un framework?

PROBLEMA A RESOLVER

En la Parte I escribimos a mano la conexión, el enrutamiento, el control de acceso y el acceso a datos. En proyectos grandes eso es mucho código repetitivo y propenso a errores.

Spring Boot

Autoconfiguración e inyección de dependencias. Spring Data (repositorios sin SQL) y Spring Security (login y acceso). Muy productivo; estándar de la industria.

Jakarta Server Faces

Enfoque basado en componentes, dentro del estándar Jakarta EE. Persistencia con JPA y transacciones con JTA gestionadas por el contenedor.

Paso a paso

- 1 **Proyecto y starters** *web, data-jpa, security, thymeleaf, postgresql*
- 2 **application.properties** *origen de datos; ddl-auto crea las tablas*
- 3 **Entidad y repositorio** *@Entity + JpaRepository (CRUD gratis)*
- 4 **Servicio y controlador** *@Service, @Controller + vista Thymeleaf*
- 5 **Spring Security** *login y control de acceso declarativos*

Entidad JPA y repositorio

Estudiante.java (entidad)

```
@Entity @Table(name = "estudiantes")
public class Estudiante {
    @Id @GeneratedValue(strategy = IDENTITY) Long id;
    private String nombre, correo, carrera; /* +get/set */ }
```

EstudianteRepositorio.java

```
public interface EstudianteRepositorio
    extends JpaRepository<Estudiante, Long> {
    List<Estudiante> findByCarrera(String carrera); // sin SQL
}
```

Controlador y vista (CRUD)

EstudianteControlador.java

```
@GetMapping
public String listar(Model m) {
    m.addAttribute("estudiantes", servicio.listar());
    return "lista";           // plantilla Thymeleaf
}
@PostMapping("/guardar")
public String guardar(@ModelAttribute Estudiante e) {
    servicio.guardar(e); return "redirect:/estudiantes"; }
```

Compáralo con EstudianteServlet de la Parte I: hace lo mismo, con mucho menos código.

La vista es Thymeleaf (th:each, th:text), que escapa por defecto — como JSTL/EL.

Login y control de acceso

SeguridadConfig.java

```
http.authorizeHttpRequests(a -> a
    .requestMatchers("/login", "/css/**").permitAll()
    .anyRequest().authenticated()           // resto protegido
    .formLogin(f -> f.loginPage("/login"));
// @Bean PasswordEncoder -> new BCryptPasswordEncoder()
```

Mira lo que desapareció

Sin AutenticacionFilter, sin gestión manual de sesión, sin SQL del CRUD. Spring lo aporta; tú declaras intenciones. Por eso era importante entender antes qué hace por debajo.

JSF con JPA y JTA

PROBLEMA A RESOLVER

Resolver el mismo problema con un enfoque basado en componentes, dentro del estándar Jakarta EE, donde el contenedor gestiona las transacciones.

- La vista se compone de componentes (Facelets .xhtml) conectados a beans gestionados.
- La persistencia se hace con JPA; las entidades usan las mismas anotaciones jakarta.persistence que en Spring.
- Las transacciones las gestiona el contenedor con JTA (@Transactional): no llamas a commit ni a rollback.
- Se ejecuta en un servidor Jakarta EE (Payara, WildFly) o en Tomcat con las extensiones.

Bean gestionado con JPA y JTA

EstudianteBean.java

```
@Named @RequestScoped           // visible como #{estudianteBean}
public class EstudianteBean {
    @PersistenceContext EntityManager em;
    @Transactional                // JTA: el contenedor abre/confirma
    public String guardar() {
        em.persist(estudiante);
        return "lista?faces-redirect=true";    // patron PRG
    }
}
```

Con @Transactional el contenedor inicia y confirma la transacción (o la deshace si hay error).

La vista con componentes (Facelets)

lista.xhtml

```
<h:dataTable value="#{estudianteBean.lista}" var="e">
  <h:column>#{e.nombre}</h:column>
  <h:column>#{e.carrera}</h:column>
</h:dataTable>
<h:form>
  <h:inputText value="#{estudianteBean.estudiante.nombre}"/>
  <h:commandButton value="Guardar"
    action="#{estudianteBean.guardar}"/>
</h:form>
```

Los componentes h: se enlazan al bean con EL (`#{...}`); `h:commandButton` invoca un método al pulsarse.

Los tres enfoques comparados

	Servlets + JSP	Spring Boot	Jakarta Faces
Enfoque	Tecnología base	Autoconfig. + inyección	Componentes (Jakarta EE)
Datos	JDBC + DAO	Spring Data JPA	JPA + JTA
Vista	JSP + JSTL/EL	Thymeleaf	Facelets (.xhtml)
Cuándo	Aprender; apps pequeñas	Productividad; industria	Empresarial; Jakarta EE

CIERRE

Recomendación de aprendizaje

- Empieza por entender bien la Parte I: te da los cimientos (el porqué de cada cosa).
- Para proyectos nuevos en la industria, Spring Boot es hoy la opción más demandada y productiva.
- Jakarta Faces es valioso si trabajarás en entornos empresariales basados en el estándar Jakarta EE.
- Lo que marca la diferencia no es la tecnología, sino los principios de diseño: capas, MVC, seguridad.

Si dominas los conceptos, cambiar de tecnología es solo aprender su sintaxis.

PARA PROFUNDIZAR

Recursos

Jakarta EE

jakarta.ee

Apache Tomcat

tomcat.apache.org

Spring Boot

spring.io/projects/spring-boot

Spring Security

spring.io/projects/spring-security

Jakarta Faces

jakarta.ee/specifications/faces

PostgreSQL

postgresql.org/docs

De los cimientos a los frameworks

Cada problema de la guía tiene ahora una solución que entiendes a fondo — y el problema completo, resuelto de tres formas.

¡Gracias!